

RPG Game Project

OOP Second Semester — SFML C++ Engine · Group Reference Document

WHAT YOU'RE BUILDING

2D RPG Game — Custom SFML C++ Engine

A fully custom RPG game in C++ using SFML for graphics and input. You build the game engine itself — no Unity, no Godot. The hero fights Dr. Vroomstein and his polymorphic minions through OOP-based class hierarchies, sprite rendering, collision detection, and a real-time combat loop.

C++ OOP SFML 2.x Inheritance Polymorphism Abstract Classes Game Loop

REQUIRED CLASSES — OOP STRUCTURE

Abstract Base

Avatar

- Fields: name, baseSpeed, maxHealth, currentHealth, position (x,y)
- Pure virtual methods: move(), attack(), render()
- Bounds-checking inside move() — clamp to screen edges

Abstract Base

Move / Moveset

- Fields: name, damage, cooldown, range, animationState
- 3 derived types: MeleeMove, RangedMove, SpecialMove
- Each type has a unique attribute (e.g. lifesteal, piercing, AoE)

Derived from Avatar

Hero (min. 3 concrete classes)

- Each hero: unique sprite, HP, speed, moveset (1 melee + 1 range + 1 special)
- S = melee, D = ranged, F = special (keyboard binding)
- Score / point system — track per move and total

Derived from Avatar

Villain — Dr. Vroomstein

- Very high HP boss character
- Randomized attack pattern (use rand() or a state machine)
- Can summon Minions mid-fight

Derived from Avatar

Minion

- Move in structured formations (e.g. V-shape, wave, line)
- Deal contact damage on collision with hero
- Different minion types per wave / level

TOOLS & SETUP YOU NEED

Library

SFML 2.6

- sfml-dev.org/download.php
- Use graphics, window, audio, system modules

Compiler

MinGW-w64 / g++

- MinGW-w64 on Windows, g++ on Linux
- Must match SFML bitness (32/64-bit)

IDE

VS Code or Code::Blocks

- VS Code + C++ extension, or Code::Blocks
- Link SFML libraries in build settings

Assets

Sprite sheets

- itch.io free tilesets or opengameart.org
- PNG format with transparent background

CORE SYSTEMS TO IMPLEMENT

- Game loop — fixed timestep update + render cycle
- `sf::Sprite` + `sf::Texture` — load and draw all characters
- Keyboard input handler — S/D/F for moves, arrow/WASD for movement
- Collision detection — AABB (axis-aligned bounding box) between hero, minions, projectiles
- Health bar UI — `sf::RectangleShape` drawn per character
- Cooldown timer — `sf::Clock` per move, blocks re-use until elapsed
- Score / points system — damage-based, displayed on HUD
- Minion formation movement — store formation offsets, update each frame
- Projectile system — spawn, move, check hit, despawn
- On-screen messages — flash text for events (hit, kill, level up)

BONUS — FOR FULL MARKS

- Multiple levels with increasing difficulty
- Difficulty modes (Easy / Normal / Hard) — affects enemy HP and speed
- Random power-ups — shield, attack boost, speed boost — spawned on map
- Animated move sprites (sprite sheet with frame cycling)
- Physics-inspired minion paths — sinusoidal or parabolic movement
- Background music + SFX via `sf::SoundBuffer`

COMPLETE ROADMAP — STEP BY STEP

1

Setup environment

Day 1 — Install SFML, link it to your IDE, run the official SFML 'Hello World' window. Confirm compiler + SFML version match. Create project folder structure: `/assets`, `/include`, `/src`.

2 Design class hierarchy on paper

Day 1–2 — Draw the UML: Avatar → Hero, Villain, Minion. Move → MeleeMove, RangedMove, SpecialMove. Write all .h files with fields and virtual functions before writing any .cpp.

3 Build game loop + single sprite on screen

Day 2–3 — Get one hero sprite moving with keyboard input. Implement bounds-checking. No combat yet — just movement and rendering.

4 Implement Move system + cooldowns

Day 3–4 — Code MeleeMove, RangedMove, SpecialMove. Bind S/D/F keys. Add sf::Clock per move. Spawn a visible projectile for ranged. No damage yet.

5 Add collision + damage

Day 4–5 — Implement AABB collision between projectiles/melee hitbox and enemies. Reduce target HP on hit. Display current HP as a rectangle bar above each character.

6 Build Villain + Minion AI

Day 5–7 — Implement Vroomstein with high HP and a random attack pattern (timer + rand() to pick attack type). Add 2–3 Minion classes with formation movement.

7 3 hero options + selection screen

Day 7–8 — Implement 3 concrete Hero classes with distinct stats and movesets. Build a simple hero selection menu using sf::Text and keyboard navigation.

8 HUD + score + on-screen messages

Day 8–9 — Render score counter, HP bars, move cooldown indicators. Flash messages on events ('CRITICAL HIT', 'MINION DEFEATED'). Use sf::Text with a loaded .ttf font.

9 Polish — animations, levels, power-ups

Day 9–12 — Add sprite sheet animation (frame cycling per move). Add 2+ levels with harder waves. Spawn random power-ups. Add difficulty selection. Optionally add sound.

10 Testing + code cleanup

Day 12–14 — Fix edge cases (out-of-bounds sprites, death handling, win/lose state). Verify all OOP requirements: abstract class, polymorphism, inheritance used correctly. Add comments to each class.

REFERENCE LINKS

GitHub	SFML + Game Dev Book github.com/SFML/SFML-Game-Development-Book — closest open-source match to this project
Website	SFML Download sfml-dev.org/download.php — official library download
Assets	Free Sprites opengameart.org or itch.io/game-assets/free
Visual	Gameplay Reference Search 'Shovel Knight gameplay' on YouTube for combat/HUD inspiration